

# Document of Week4 Assessment

## Selenide and Pact Contract Testing

### Introduction

This submission contains two independent automation projects.

1. **Selenide UI Automation**
  2. **Pact Contract Testing**
- 

### Project 1 : Selenide UI Automation

#### Objective

The objective of this project is to automate the Product Catalog application using Selenide by implementing the Page Object Model (POM). The project validates different user journeys such as searching products, opening product details, adding products to the cart, and verifying product information.

---

#### Technology Stack

- Java 22
  - Maven
  - Selenide
  - JUnit 5
  - IntelliJ IDEA
  - Chrome Browser
- 

#### Project Structure

Test→>java→>selenide→

Config→Config.java

Pages→SelenideProductPage.java

## **Configuration**

The Config class stores the browser configuration.

```
Configuration.baseUrl = "http://localhost:5173";
```

```
Configuration.browser = "chrome";
```

```
Configuration.browserSize = "1440x900";
```

```
Configuration.headless = false;
```

This configuration launches the Chrome browser with the specified window size.

---

## **Page Object Implementation**

The page class contains all web element locators and reusable methods.

Examples include

- Open Catalog
- Search Product
- Click Product
- Verify Product Title
- Verify Product Availability
- Verify Brand
- Verify Rating
- Verify Price

Using Page Object Model keeps the test code clean and reusable.

---

## **Test Scenarios**

### **Test Case 1**

Verify Catalog page opens successfully.

Expected Result

Product Catalog page is displayed.

---

### **Test Case 2**

Search for "Running"

Expected Result

Matching products should be displayed.

---

### **Test Case 3**

Verify Product Details

Flow

Catalog

↓

Search Product

↓

Open Product Details

↓

Verify Product Title

---

### **Test Case 4**

Verify Product Availability

Flow

Catalog

↓

Search Product

↓

Open Details

↓

Verify Availability

Expected Result

---

### **Test Case 5**

Verify Brand

Expected Result

Correct brand name should be displayed.

---

### **Test Case 6**

Verify Product Price

Expected Result

Displayed price should be correct.

---

### **Test Case 7**

Verify Invalid Search

Search

Laptop123

Expected Result

No products should be displayed.

---

### **Selenide Collection Methods Used**

#### **filter()**

Filters elements matching a condition.

---

#### **filterBy()**

Filters collection using Selenide conditions.

```
results().filterBy(text("Running Shoes"))
```

---

#### **shouldHave(size())**

Verifies number of elements.

---

### **shouldHave(texts())**

Verifies all displayed product names.

---

### **Commands Used**

Clean Project

mvn clean

Compile

mvn compile

Run Tests

mvn test

Run Single Test

mvn -Dtest=SelenideProductTest test

---

### **Learning Outcome**

- Page Object Model
  - Element Collections
  - Assertions
  - Selenide Conditions
- 

## **Project 2 : Pact Contract Testing**

### **Objective**

The objective of this project is to verify communication between the Consumer and Provider without deploying both applications together.

Consumer creates the contract.

Provider verifies the contract.

This ensures API compatibility before deployment.

---

## Technology Stack

- Java
  - Maven
  - Pact JVM
  - Rest Assured
  - WireMock
  - Docker
  - Pact Broker
  - GitHub Actions
- 

## Project Structure

Src→main→test

→pact→ OmsClient.java  
→ PosOmsConsumerPactTest.java  
→ OmsProviderPactTest.java  
→ resources→docker-compose.pact-broker.yml

---

## Consumer

The consumer interacts with the OMS API using Rest Assured.

Methods include

- Get Order
  - Create Order
  - Get Inventory
  - Get Forbidden Order
  - Get Missing Order
- 

## Provider

The provider uses WireMock.

Each provider state returns the response expected by the consumer.

#### Provider States

- Order exists
- Creating Order
- Inventory Available
- Forbidden
- Order Not Found

---

### Positive Scenarios

#### Get Order

GET /order/123

Status

200

---

#### Create Order

POST /orders/

Status

201

---

### Inventory

GET /inventory/SKU-9

Status

200

---

### Negative Scenarios

#### 403 Forbidden

Purpose

User is not authorized to access the resource.

GET /order/999

Status

403

Response

```
{  
  "message": "Forbidden"  
}
```

---

## **404 Not Found**

Purpose

Requested order does not exist.

GET /order/404

Status

404

Response

```
{  
  "message": "Order not found"  
}
```

---

## **Pact Workflow**

Consumer Test

↓

Pact File Generated

↓

Publish to Pact Broker

↓



Provider Verification

↓

Can-I-Deploy Check

↓

Deployment

---

## **Docker Commands**

Start Broker

```
docker compose -f src/test/resources/docker-compose.pact-broker.yml up -d
```

Stop Broker

```
docker compose -f src/test/resources/docker-compose.pact-broker.yml down
```

Restart

```
docker compose -f src/test/resources/docker-compose.pact-broker.yml down -v
```

```
docker compose -f src/test/resources/docker-compose.pact-broker.yml up -d
```

Check Running Containers

```
docker ps
```

---

## **Maven Commands**

Clean

```
mvn clean
```

Compile

```
mvn compile
```

Run Consumer Tests

```
mvn -Dtest=PosOmsConsumerPactTest test
```

Run Provider Tests

```
mvn -Dtest=OmsProviderPactTest test
```

Run All Tests

```
mvn test
```

---

## Git Commands

Clone Repository

```
git clone <repository-url>
```

Create Branch

```
git checkout -b feature/pact-testing
```

Check Status

```
git status
```

Stage Changes

```
git add .
```

Commit

```
git commit -m "Implemented Pact consumer and provider tests"
```

Push Branch

```
git push origin feature/pact-testing
```

---

## CI/CD Workflow

1. Checkout Repository
  2. Setup Java
  3. Install Pact CLI
  4. Start Pact Broker
  5. Execute Consumer Tests
  6. Publish Contracts
  7. Execute Provider Verification
  8. Record Deployment
  9. Run Can-I-Deploy
  10. Complete Build
- 

## Conclusion

Both projects demonstrate different aspects of automation testing. The Selenide project validates the application's user interface through end-to-end browser automation, while the Pact project ensures reliable communication between services through consumer-driven contract testing. Together, these projects improve software quality by detecting UI defects and API compatibility issues early in the development lifecycle.